



UNIVERSIDAD
MAYOR
para espíritus emprendedores

INFORME DE PROYECTO UNIVERSITARIO

Repuestos Exactos

Arquitectura de desarrollo móvil y web para el diagnóstico automotriz OBD-II y la cotización optimizada de repuestos mediante recomendación analítica.



UNIVERSIDAD MAYOR

Facultad de Ciencias, Ingeniería y Tecnología — Ingeniería Civil en Computación e Informática

DATOS DEL TRABAJO

Autor(es): Javier Chandía - Matias Daza - Victoria Escobar- Adrián Perugini

Ciudad(es): Santiago/ Rancagua

Docente / Tutor: Carlos Muñoz

Asignatura: Arquitectura de desarrollo (Móvil y Web) | INGT:1003-001

Fecha: 21/09/2026

i Índice

Índice	2
Introducción	3
Objetivos	4
2.1 Objetivo general	4
2.2 Objetivos Específicos	4
Antecedentes	5
Marco Teórico	6
Metodología	7
Arquitectura cliente servidor y capas	8
6.1 Capas del modelo	8
6.2 Estructura interna	9
Selección de lenguajes y frameworks	10
Resultados Esperados	11
Conclusiones	12
Referencias	13
Anexos	14

1 Introducción

En la industria automotriz actual, el mantenimiento y la reparación de vehículos representan un reto tanto para propietarios individuales como para profesionales. A medida que la arquitectura electrónica de los automóviles se vuelve más compleja, la detección de averías mediante herramientas de diagnóstico como (OBD-II) se ha transformado en un estándar operativo indispensable de los autos fabricados a partir de 1990 con el primer modelo de OBD, luego con la segunda generación de scanners (OBD-II) que funcionan en la mayoría de los vehículos fabricados después de 1996 según la revista “MotorTrend” (MotorTrend, 2024, #). “Ahora podíamos tener un componente dentro de los valores de rango correcto, sin embargo, la lectura de valores en tiempo real nos daba pistas clave para localizar una causa de avería. Hablamos ya del año 2000” (Autodemiaonline, n.d.). Sin embargo, existe un problema crítico en la fase posterior al diagnóstico: la adquisición del repuesto o componente adecuado.

Actualmente, el proceso que transcurre desde la lectura de un código de falla hasta la instalación de la pieza correcta está distorsionado, es poco claro y consume tiempo significativo. Los usuarios se enfrentan a un mercado de repuestos saturado, con profundas desigualdades de información en cuanto a precios, compatibilidad, origen de los componentes y calidad de los fabricantes. Esto provoca decisiones de compra ineficientes, donde a menudo se adquieren repuestos de baja durabilidad para ahorrar costos, o bien se pagan sobrepagos injustificados por falta de alternativas viables.

El presente proyecto plantea el desarrollo de una solución en forma de aplicación móvil conectada a escáneres automotrices (OBD-II). La plataforma no solo interpretará las fallas mecánicas y electrónicas en tiempo real, sino que optimizará el proceso de cotización de componentes. A través de un algoritmo de recomendación analítico, la aplicación evaluará múltiples marcas y distribuidores, ofreciendo al usuario una matriz comparativa basada en criterios de costo, practicidad, durabilidad y riesgos potenciales asociados a repuestos económicos. De este modo, la herramienta busca democratizar la información técnica y comercial, transformando un diagnóstico complejo en una toma de decisiones informada, ágil y eficiente.

2 Objetivos

2.1 Objetivo general

Desarrollar una aplicación móvil orientada al diagnóstico automotriz y cotización de repuestos, mediante la integración de escáner OBD-II y un algoritmo de recomendaciones analíticas con el fin de automatizar la toma de decisiones de la adquisición de componentes según criterios de costo, durabilidad y compatibilidad.

2.2 Objetivos Específicos

- ▶ **Integración Diagnóstica y Protocolos de Comunicación:** Diseñar un módulo de software capaz de vincularse vía Bluetooth/Wi-Fi con dispositivos de escaneo automotriz (protocolos OBD-II) para extraer e interpretar códigos de falla (DTC) de manera precisa y universal.
- ▶ **Motor de Cotización Multifuente y Mapeo de Vehículos:** Desarrollar un módulo de consulta de precios mediante una arquitectura híbrida: un sistema de identificación vehicular con lectura automática de VIN (vía OBD-II) y respaldo por selección manual de parámetros (Marca, Modelo, Año, Motor); integrado a un motor de búsqueda que consulte la API de e-commerce consolidados (como Mercado Libre) e índices locales de precios (base de datos relacional) para categorizar repuestos compatibles en tiempo real.
- ▶ **Algoritmo de Evaluación y Clasificación de Componentes:** Programar un sistema de recomendación analítico que categorice los repuestos según tres perfiles principales:
 - Perfil económico: Menor costo inicial, advirtiendo fallas potenciales o menor vida útil.
 - Perfil Práctico/Equilibrado: Mejor relación precio-calidad basada en rendimiento comprobado.
 - Perfil Óptimo/OEM: Piezas originales o de calidad equivalente para máxima durabilidad.
- ▶ **Diseño de Interfaz de Usuario (UI/UX) Formal e Intuitiva:** Crear una interfaz móvil clara que traduzca el lenguaje técnico automotriz a reportes comprensibles para el usuario final, mostrando comparativas transparentes de precios, garantías y riesgos de cada componente.
- ▶ **Validación y Pruebas de Campo:** Realizar pruebas piloto en entornos controlados (talleres aliados y usuarios finales) para validar la precisión de las cotizaciones automatizadas, la estabilidad de la conexión con el escáner y la tasa de satisfacción en la toma de decisiones de compra.

3 Antecedentes

El siempre creciente tamaño del parque automotor chileno arrastra consigo otra demanda implícita: una necesidad permanente de mantenimiento, reparación y reposición de componentes. Según el Instituto Nacional de Estadísticas (INE, 2026), el parque de vehículos motorizados creció un 2,9 % entre 2023 y 2024. Por otro lado, el crecimiento del comercio electrónico ha ampliado considerablemente la cantidad de alternativas disponibles para los consumidores. Si bien esto facilita el acceso a productos y proveedores, también traslada al usuario la responsabilidad de comparar precios, disponibilidad, condiciones de entrega y otras variables antes de tomar una decisión de compra. Esta situación adquiere relevancia en el mercado de repuestos automotrices, donde un mismo componente puede encontrarse en múltiples tiendas y plataformas, con diferencias significativas de precio y disponibilidad. En consecuencia, encontrar una alternativa conveniente no depende únicamente de que exista una oferta, sino de que el usuario pueda acceder y comparar la información disponible en distintos canales.

Con el objetivo de conocer la ruta del usuario, decidimos hacer una prueba de comparación: comparamos el precio de un mismo componente en distintas tiendas. Para ello, seleccionamos como ejemplo un Filtro de Aceite Original de Toyota (OEM:90915-YZZE1) y comparamos su precio en diversas tiendas: Vigfor, Mercadolibre, AliExpress, DQ Motors y Comercial Miraflores (véase Anexo C). Los resultados evidenciaron una diferencia del 117% entre el precio más bajo y el más alto, correspondientes a Vigfor y AliExpress, respectivamente (Vigfor, s. f.; AliExpress, s. f.; véase Anexo C). Además, dentro de la muestra analizada se observó una variación de \$3.314 entre los precios. Si seguimos la ruta que debe recorrer un usuario para adquirir este producto, surge una pregunta: ¿Cómo puedo saber si estoy pagando el mejor precio disponible? Incluso después de comparar algunas alternativas, permanece esa incertidumbre: ¿Existirá otra tienda o plataforma que no estoy considerando y que ofrezca el producto a un precio aún menor?.

Esta situación evidencia una de las dificultades que buscamos abordar: la falta de una forma sencilla de comparar precios entre distintos proveedores y encontrar la alternativa más conveniente para un mismo repuesto. Históricamente se han realizado múltiples propuestas similares que intentan abordar este mismo problema, pero en el mercado chileno, el estado del arte se encuentra discontinuado o sin ninguna opción que domine a las otras. El competidor más cercano actual es ComparaRepuestos, web que permite ingresar a los usuarios los datos de su vehículo, el repuesto que se busca y a partir de esa información la página cotiza en varias tiendas registradas. Sin embargo, un limitante de esta web es su propia fortaleza: estar limitado a piezas intercambiables (espejos,puertas, etc.). Por lo que el daño físico ya debe estar hecho de antemano. Encontramos aquí dos propuestas de valor para nuestra página; Ayudar a los clientes a identificar posibles soluciones al problema que enfrenta su vehículo y qué repuestos pueden solucionar esta problemática, considerando cuál de las opciones es más accesible y encontrar las piezas a mejor precio. El usuario sabe que su vehículo tiene un problema, pero no necesariamente sabe qué lo provoca, qué pieza necesita, cuál es compatible con su vehículo ni dónde comprarla al mejor precio. Nosotros intentamos llenar esos vacíos.

4 Marco Teórico

Para el desarrollo de un sistema de diagnóstico vehicular basado en un dispositivo OBD-II, es necesario comprender los estándares que regulan la comunicación entre el vehículo y las herramientas de escaneo, así como los protocolos de transmisión de datos que permiten interpretar la información generada por la unidad de control del motor. A continuación, se describen los principales marcos normativos que sustentan el funcionamiento técnico del proyecto.

Diagnóstico a bordo (OBD) y protocolos de comunicación(CAN bus)

El diagnóstico a bordo (OBD) regula las interacciones entre las herramientas de escaneo y los sistemas del vehículo. La norma SAE J1978-1 especifica las funciones de las herramientas OBD-II para garantizar un acceso seguro a los servicios de la norma SAE J1979 bajo la regulación de EE.UU. (SAE International, 2026)

- **Funciones Principales:** Define la comunicación vehículo-escáner, la autodeterminación del protocolo para aplicaciones SAE J1979, J1979-2, y J1979-3, y los servicios de primera generación (SAE International, 2026)
- **Alcance:** La J1978 distingue los protocolos de primera y segunda generación. Permite funciones adicionales siempre que no ponga en riesgo la red del vehículo ni la herramienta. (SAE International, 2026)

Red CAN bus de datos e intercambio de Datos

La norma ISO 11898-1:2015 rige la capa de enlace de datos de la red “Controller Area Network” (CAN) bajo el modelo OSI, detallando las subcapas LLC, MAC y PCS para la comunicación digital entre unidades de control electrónico (*ISO 11898-1:2015 - Road Vehicles — Controller Area Network (CAN) — Part 1: Data Link Layer and Physical Signalling*, n.d.)

- **CAN Clásico:** Soporta hasta 1 Mbit/s y cargas de 8 bytes por trama (*ISO 11898-1:2015 - Road Vehicles — Controller Area Network (CAN) — Part 1: Data Link Layer and Physical Signalling*, n.d.)
- **CAN FD (Flexible Data-Rate):** Supera 1 Mbit/s y cargas de más de 8 bytes por trama (*ISO 11898-1:2015 - Road Vehicles — Controller Area Network (CAN) — Part 1: Data Link Layer and Physical Signalling*, n.d.)
- **Implementación:** Ofrece tres opciones de compatibilidad, recomendando la adopción conjunta de CAN Clásico y CAN FD para nuevos desarrollos (*ISO 11898-1:2015 - Road Vehicles — Controller Area Network (CAN) — Part 1: Data Link Layer and Physical Signalling*, n.d.)

Identificación Vehicular

La norma ISO 3779:2009 establece la estructura uniforme a nivel mundial del Número de Identificación Vehicular (VIN) para garantizar la trazabilidad legal y técnica de automotores, remolques y motocicletas (International Organization for Standardization, 2009)

El conocimiento de estas normas es fundamental para el desarrollo del proyecto, ya que garantiza que la herramienta propuesta sea compatible con los protocolos estándar de la industria automotriz (OBD-II y CAN bus), permitiendo una lectura confiable de los códigos de falla y parámetros del vehículo. Asimismo, la identificación vehicular mediante el VIN (ISO 3779:2009) posibilita, en etapas futuras, asociar el diagnóstico a las características específicas de cada vehículo, aportando mayor precisión al sistema. Estos fundamentos técnicos respaldan la viabilidad del proyecto y su alineación con los estándares vigentes en la industria.

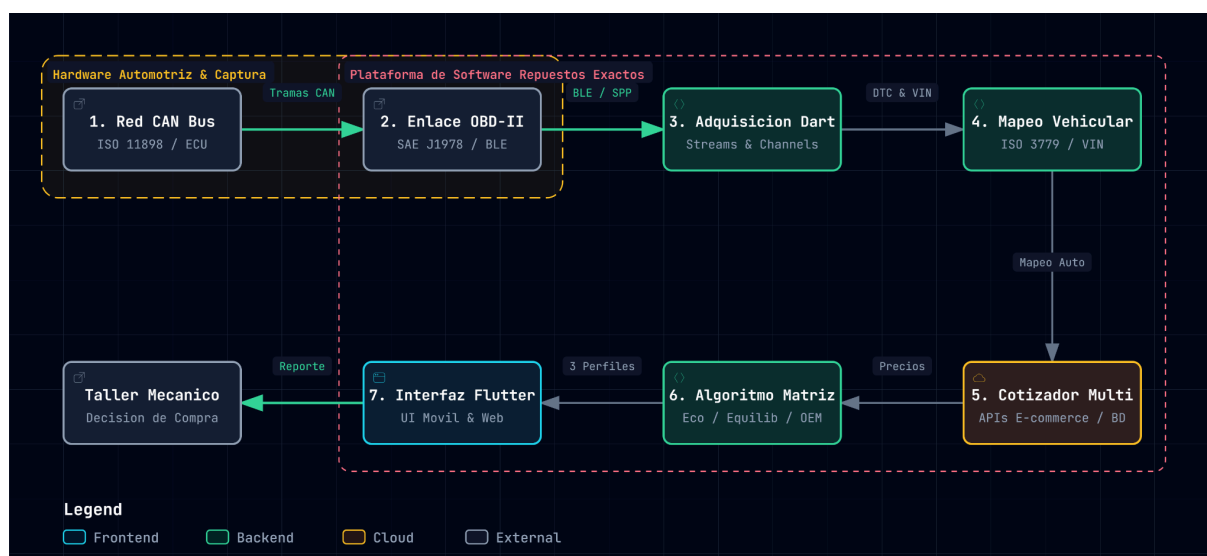
5 Metodología

El proyecto se abordó como un desarrollo tecnológico aplicado, de carácter exploratorio, organizado en etapas iterativas. Primero nos pusimos en el rol de mecánicos del rubro automotriz o clientes que buscan repuestos, esto con el objetivo de identificar sus necesidades y definir los requerimientos funcionales de la herramienta. Luego se definió la identidad visual del proyecto (nombre, logotipo, paleta de colores y lineamientos de diseño). En paralelo, se revisaron normas y documentación técnica (SAE J1978-1, ISO 11898-1, ISO 3779 y documentación oficial de Flutter y React Native). Para dimensionar el problema de la cotización, el 7 de septiembre de 2026 se comparó el precio publicado del filtro de aceite Toyota OEM 90915-YZZE1 en cinco sitios (Mercadolibre, AliExpress, Vigfor, DQMotors y Comercial Miraflores), registrando los datos en el Anexo C. Para la elaboración del producto se diseñó una arquitectura por capas, tomando el modelo OSI como referencia para el flujo de datos y definiendo una estructura interna con interfaz, módulos de diagnóstico, búsqueda e historial, un núcleo de dominio, un cliente BLE, un cotizador y persistencia offline (sección 6). Se compararon dos alternativas, Flutter/Dart y React Native/JavaScript, según manejo de flujos asíncronos, acceso a Bluetooth, rendimiento y experiencia previa del equipo (sección 7). Finalmente, se adquirió un dispositivo OBD-II [modelo] y se desarrolló una aplicación web de prueba para leer y visualizar códigos de falla y parámetros del motor en tiempo real, mientras el resto del equipo elaboró el informe. Dado el alcance de esta fase, el resultado es un prototipo de prueba de concepto; la consulta a mecánicos y la comparación de precios tienen carácter exploratorio y no representan una muestra generalizable.

6 Arquitectura cliente servidor y capas

La arquitectura del sistema Repuestos Exactos se estructuró bajo un enfoque por capas lógicas de software, asegurando una clara separación de responsabilidades, alta mantenibilidad y bajo acoplamiento entre la captura de hardware, la lógica de negocio y la experiencia de usuario.

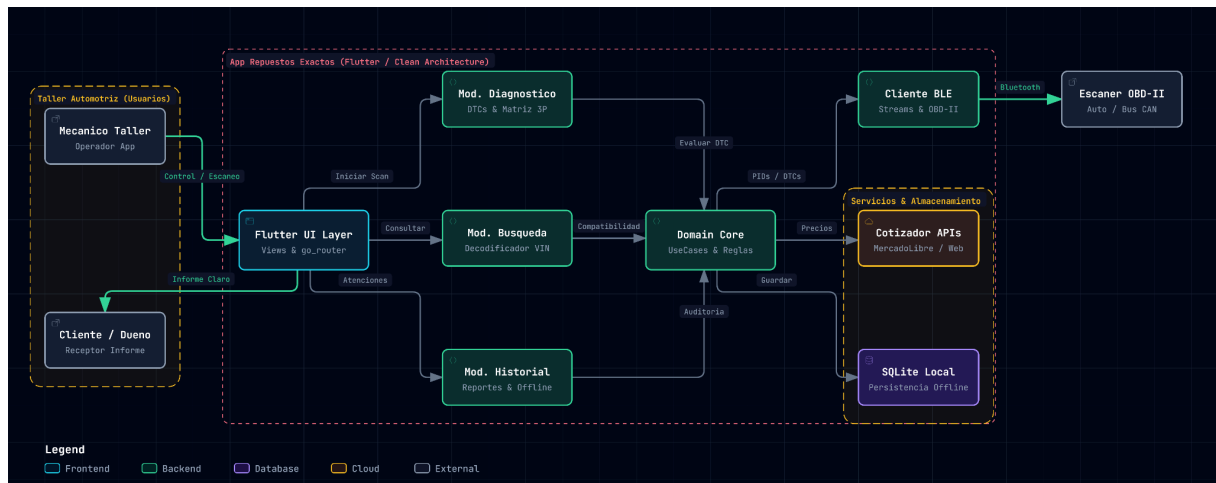
6.1 Capas del modelo



Usando el Modelo de Capas OSI:

- Capa 1: (Capa física) : El escáner físico (OBD-II) es el dispositivo que se usa para analizar el código del auto.
- Capa 2: (Enlace de datos): El código entregado por el OBD-II, que será enviado a la aplicación vía bluetooth.
- Capa 3: (Capa de red): Administra los datos, el código en este caso, se envía y llega correctamente a la aplicación.
- Capa 4: (Capa de transporte): Se encarga de transmitir la información de extremo a extremo, y segmentar la información. Un mapeo vehicular para buscar el componente solicitado.
- Capa 5(Sesión / Nube): Se encarga de administrar los datos, guardarlos y obtener la información solicitada.
- Capa 6: Se encarga de traducir los datos codificados y solicitados, para que la capa 7 pueda visualizarlos.
- Capa 7 (Aplicación): Es como se presenta la app visualmente, como se presenta al cliente / el color / diseño / arquitectura.

6.2 Estructura interna



- **Mecánico taller:** El cliente/Mecánico el cual está usando la aplicación
- **Flutter UI:** Es el punto central de interacción visual. Recibe las órdenes del mecánico y distribuye las acciones hacia tres módulos principales de la aplicación dependiendo de lo que se necesite hacer
- **Mod. Diagnóstico (DTCs & Matriz 3P):** Se activa al "Iniciar Scan". Se encarga de procesar los códigos de falla (DTCs) detectados en el vehículo.
- **Mod. Búsqueda (Decodificador VIN):** Se activa al "Consultar". Su función es decodificar el número de identificación del vehículo (VIN) para asegurar la compatibilidad de las piezas.
- **Mod. Historial (Reportes & Offline):** Administra el registro de las atenciones previas, permitiendo que la información esté disponible para generar reportes incluso sin conexión a internet.
- **Domain Core (Use Cases & Reglas):** Es el "cerebro" del sistema. Aquí residen todas las reglas de negocio. Centraliza la evaluación de códigos del módulo de diagnóstico y la compatibilidad del módulo de búsqueda. Luego, decide qué hacer con esa información (por ejemplo, buscar precios o guardar registros).
- **Cliente BLE (Streams & OBD-II):** Es el componente de software dentro de la app encargado de mantener la conexión Bluetooth de baja energía (BLE). Recibe el flujo continuo de datos (PIDs / DTCs) y se los entrega al *Domain Core*.
- **Escáner OBD-II (Auto / Bus CAN):** Es el hardware físico conectado al puerto del vehículo. Lee la información directa de la computadora del auto y la transmite vía Bluetooth al Cliente BLE.
- **Cotizador APIs (MercadoLibre / Web):** El *Domain Core* le envía peticiones ("Precios") a este módulo, el cual se conecta a internet para consultar el valor en tiempo real de los repuestos necesarios.
- **Persistencia Offline:** Una base de datos local (probablemente SQLite u otra solución local) donde el *Domain Core* ejecuta acciones de "Guardar" y "Auditoria" para asegurar que ningún diagnóstico se pierda si el taller se queda sin internet.

7 Selección de lenguajes y frameworks

Opción 1

- Lenguaje de programación: Dart
- Frameworks: Flutter
- Justificación:
 - **Gestión de Datos Asíncronos en Tiempo Real mediante Streams:** El diagnóstico vía OBD-II (Bluetooth) no entrega un dato estático, sino un flujo continuo de datos de telemetría y eventos en segundo plano. Dart resuelve esta necesidad mediante el modelo de Stream, definido como: “Un flujo es una secuencia de eventos asíncronos. Es como un iterable asíncrono, donde, en lugar de obtener el siguiente evento cuando se solicita, el flujo indica que hay un evento cuando está listo”(Dart, 2025). Esto nos permite que la aplicación lea los datos del escáner en tiempo real de forma reactiva y gestione de manera nativa los eventos de error. (Dart, 2025)
 - **Comunicación con Hardware y Canales de Plataforma :** Para interactuar con los sockets y controladores Bluetooth del dispositivo, Flutter permite la comunicación nativa sin perder rendimiento mediante canales de plataforma e interfaces externas: “Para aplicaciones móviles y de escritorio, Flutter permite llamar a código personalizado a través de un canal de plataforma, que es un mecanismo para comunicar entre tu código Dart y el código específico de la plataforma de tu aplicación host” (*Flutter Architectural Overview*, n.d.). Para las API basadas en C, Dart proporciona un mecanismo directo para vinculación nativa mediante la biblioteca dart:ffi (*Flutter Architectural Overview*, n.d.). Esta capacidad asegura la recepción fluida de códigos de error (DTC) desde el bus CAN del vehículo directamente al núcleo de la aplicación. (*Flutter Architectural Overview*, n.d.)

Opción 2

- Lenguaje de programación: JavaScript
- Frameworks: React Native
- Justificación:
 - **Arquitectura Basada en Componentes Nativos y Comunicación Asíncrona (Bridge):** A diferencia de las soluciones web embebidas, React Native no compila JavaScript a código Java u Objective C, ni tampoco transforma la lógica a un bytecode propio del sistema operativo (*Stack Overflow*, n.d.) En su lugar, ejecuta el código JavaScript en un hilo independiente y renderiza componentes de la interfaz de usuario directamente nativos (UIView en iOS o android.view.View en Android) (Franchet, 2017). Esta separación entre el entorno de ejecución de JavaScript y el hilo nativo de la plataforma se gestiona mediante un puente (Bridge), una interfaz desarrollada en C/C++ que actúa como intermediario de mensajes

(Franchet, 2017).

- **Intercambio de Datos y Procesamiento de Eventos de Telemetría (OBD-II):** Cuando el escáner del auto envía datos sin parar a través de Bluetooth, la aplicación necesita procesarlos sin ponerse lenta. React Native resuelve esto separando el código en dos partes que se comunican mediante un puente (Bridge). Como explica Franchet (2017): “En React Native, el puente asume la función de intermediario de mensajes, gestionando los comandos asíncronos entre los dos mundos diferentes”. Esta comunicación en segundo plano evita interrupciones en la pantalla, ya que “al ser asíncrono, no es bloqueante y, por lo tanto, permite una gestión fluida de la vista en la pantalla” (Franchet, 2017). Gracias a esto, la telemetría del vehículo se actualiza en tiempo real sin congelar la interfaz de la aplicación.

8 Resultados Esperados

Se espera que, al finalizar el proyecto, el equipo cuente con un prototipo funcional de aplicación web capaz de conectarse a un dispositivo OBD-II y desplegar en tiempo real los parámetros y códigos de falla del vehículo, validando la comunicación entre hardware y software como prueba de concepto. La interfaz web que permita la lectura de datos del vehículo, con una interfaz de usuario coherente a la identidad visual definida por el equipo (color morado, logo, nombre), orientada a facilitar la interpretación de la información técnica por parte del usuario final. A nivel académico, se espera consolidar el aprendizaje del equipo en el desarrollo de soluciones tecnológicas orientadas a un problema real, integrando conocimientos de programación, arquitectura de desarrollo, diseño de interfaces y trabajo con hardware embebido.

9 Conclusiones

El proyecto Repuestos Exactos permitió cumplir una primera fase orientada a validar la viabilidad técnica y la necesidad de mercado de la solución propuesta. Se desarrolló un prototipo web de prueba de concepto capaz de conectarse a un dispositivo OBD-II y [leer y visualizar en tiempo real códigos de falla y parámetros del vehículo], lo que confirma la comunicación entre hardware y software. Además, se definieron los requerimientos funcionales a partir de la consulta a mecánicos, la identidad visual del proyecto y una arquitectura por capas, junto con la comparación de dos alternativas tecnológicas (Flutter/Dart y React Native/JavaScript).

La comparación de precios del filtro de aceite Toyota OEM 90915-YZZE1 en cinco sitios evidenció una diferencia de 117% entre el precio más bajo y el más alto, lo que respalda la necesidad de una herramienta que facilite la cotización. Este ejercicio fue exploratorio, con un solo producto, una fecha y cinco tiendas, por lo que sus resultados no son generalizables.

Quedan pendientes para las siguientes fases el motor de cotización multifuente, el algoritmo de recomendación por perfiles, la decodificación del VIN, el desarrollo de la aplicación móvil y las pruebas piloto con talleres, correspondientes a los objetivos específicos 2, 3 y 5.

Durante el desarrollo, el equipo comprendió la importancia de estandarizar los códigos OEM para comparar el mismo producto entre catálogos heterogéneos, y el valor de diseñar interfaces que traduzcan información técnica automotriz en datos comprensibles para el usuario. Asimismo, se fortalecieron las competencias prácticas en la integración de hardware OBD-II con software.

10 Referencias

- Autodemiaonline. (n.d.). *Evolución de la Diagnóstico del Automóvil*. Autodemiaonline. <https://autodemiaonline.com/evolucion-de-la-diagnosis-del-automovil/>
- Instituto Nacional de Estadísticas. (2026, 4 de febrero). *INE publica edición 2025 del Informe Anual de Estadísticas del Medio Ambiente*. <https://www.ine.gob.cl/sala-de-prensa/prensa/general/noticia/2026/02/04/ine-publica-edici%C3%B3n-2025-del-informe-anual-de-estad%C3%ADsticas-del-medio-ambiente>
- International Organization for Standardization. (2009, 10 10). *ISO 3779:2009 - Road vehicles — Vehicle identification number (VIN) — Content and structure*. ISO. <https://www.iso.org/standard/52200.html>
- *ISO 11898-1:2015 - Road vehicles — Controller area network (CAN) — Part 1: Data link layer and physical signalling*. (n.d.). ISO. Retrieved September 2, 2026, from <https://www.iso.org/standard/63648.html>
- MotorTrend. (2024, 08 26). *What Is an OBD2 Scanner and How Does It Work?* MotorTrend. <https://www.motortrend.com/how-to/what-is-an-obd2-scanner-how-it-works>
- SAE International. (2026, 04 20). *SAE Movilus. Herramienta de diagnóstico OBD-II: Protocolos de primera generación*. https://saemobilus.sae.org/standards/j1978-1_202604-obd-ii-scan-tool-first-generation-protocols
- Dart. (2025, 10 18). *Asynchronous programming: Streams*. Dart. <https://dart.dev/libraries/async/using-streams>
- *Flutter architectural overview*. (n.d.). Flutter documentation. Retrieved September 7, 2026, from <https://docs.flutter.dev/resources/architectural-overview>
- Franchet, M. (2017, 11 10). *Hackernoon*. Comprender el concepto de puente de React Native. <https://hackernoon.com/understanding-react-native-bridge-concept-e9526066ddb8>
- *Stack Overflow*. (n.d.). Does React Native compile JavaScript into Java for Android? Retrieved September 14, 2026, from <https://stackoverflow.com/questions/41124338/does-react-native-compile-javascript-in-to-java-for-android>
- AliExpress. (s. f.). *Filtro de aceite original Toyota 90915-YZZN2 90915YZZN2 Referencia australiana: z386 90915-YZZE1*. Recuperado el 7 de Septiembre de 2026, de <https://es.aliexpress.com/item/1005011560505631.html>
- Comercial Miraflores. (s. f.). *Filtro aceite original Toyota W 68/80 W 68/3 EQ 90915-YZZE1*. Recuperado el 7 de Septiembre de 2026, de <https://comercialmiraflores.cl/producto/filtro-aceite-original-toyota-w-68-80-w-68-3-eq-90915-yzze1/>
- DQMotors. (s. f.). *Filtro de aceite original Toyota 90915-YZZE1*. Recuperado el 7 de Septiembre de 2026, de <https://dqmotors.cl/producto/filtro-de-aceite-original-toyota-90915-yzze1/>
- Mercado Libre. (s. f.). *Filtro aceite original Toyota Hilux Yaris Corolla y más*. Recuperado el 7 de Septiembre de 2026, de

<https://www.mercadolibre.cl/filtro-aceite-original-toyota-hilux-yaris-corolla-y-mas/up/MLCU3445378822>

- Vigfor. (s. f.). *Filtro aceite Toyota Yaris Celica 1.5 1.8*. Recuperado el 7 de Septiembre de 2026, de <https://vigfor.cl/filtro-aceite-toyota-yaris-celica-1-5-1-8/201000>

11 Anexos

Anexo A: Declaración de Uso de Inteligencia Artificial

En conformidad con las exigencias académicas e institucionales del proyecto, el equipo de trabajo declara que se empleó la herramienta de Inteligencia Artificial Generativa Gemini (Google) como apoyo exclusivo en la fase de investigación, específicamente para la búsqueda, rastreo y consolidación de páginas web de información.

Anexo B: Repositorio en GitHub

El código fuente del proyecto, la estructura de carpetas en Flutter, los módulos de lectura BLE y la integración de servicios se encuentran alojados y disponibles para auditoría técnica en el siguiente repositorio público:

- <https://github.com/Risu-31/Repuestos-Exactos>

Anexo C: Registro de búsqueda y comparación de precios.

La siguiente tabla presenta el registro de precios obtenidos mediante la búsqueda y comparación del producto Filtro de Aceite Original Toyota 90915-YZZE1 con fecha 7 de septiembre de 2026 en distintos sitios de comercio electrónico y distribuidores de repuestos. Los precios corresponden a los valores observados al momento de realizar la búsqueda.

N.º	Sitio Consultado	Producto	Precio Observado (CLP)
1	Vigfor	Filtro de aceite Toyota 90915-YZZE1	\$7.144
2	MercadoLibre	Filtro de aceite original Toyota 90915-YZZE1	\$10.850
3	AliExpress	Filtro de aceite Toyota 90915-YZZE1	\$15.500
4	DQMotors	Filtro de aceite original Toyota 90915-YZZE1	\$9.990
5	Comercial Miraflores	Filtro de aceite original Toyota 90915-YZZE1	\$14.000